LoRA4VR — Training Log

A living record of experiments, findings, and open questions

What this is. A dated log of experiments run on the LoRA4VR multi-turn RL loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result. The paths `docs/rung0/` and `scripts/rung0_gate_crystal.py` named in entries up to 2026-10-25 are now `docs/stage0/` and `scripts/stage0_gate_crystal.py`, which was replaced on 2026-11-01 by `scripts/stage0_gate.py`; a level of the experiment plan is called a stage.

System. `Qwen3-4B-Instruct-2507` with a LoRA adapter trained by branched multi-turn GRPO (`src/branched_trainer.py`). The model writes `policy(state) -> action` Python for a seeded cooling crystallizer with a hidden metastable limit (`scripts/seed_tasks/crystallizer_env.py`); the final run is scored in a sandboxed subprocess, after an optional unscored probe run on the same environment. The two Stage 0 entries dated 2026-10-24 concern the earlier slosh-tank environment, which was dropped that day. Method and experimental design: `docs/paper.tex`.

Experiment plan and status. The plan is a ladder of stages; each stage is a go-or-stop check for the next, and the dated entries below hold the evidence. This table is kept current as stages complete; the entries themselves are never edited. The full plan with pass criteria is the section *Experiment plan* of `docs/paper.tex`.

Stage	What it asks	Status	Entry
0	Is the environment worth it: does the hidden limit need a probe run?	passed on the current environment (clean rebuild, run 2); earlier passes were on the previous environment	2026-11-01, Stage 0 rebuilt
0b	Does the hand-written probe run + final run survive the real runner?	folded into Stage 0 as gate G3 on 2026-11-01; passed	2026-11-01, Stage 0 rebuilt
1	Frozen model, one turn, no probe run (arm A)	re-run on the 48-task set with the 4B base and Qwen3-8B (non-thinking); both far below the cautious blind reference, the 8B worse than the 4B	2026-11-01, Stages 1 and 2 re-run
2	Frozen model with a probe run (arm B)	re-run as above; about six percent of first replies are probe code and every one is cut off by the 1000-token budget, so no probe runs	2026-11-01, Stages 1 and 2 re-run
3	Single-turn GRPO (arm C)	pending	
4	Flat multi-turn training, StarPO (arm D)	pending	
5	Branched rollouts, flat advantage (arm E)	pending	
6	Branched rollouts, per-turn baselines, ours (arm F)	pending	

2026-10-24 Log reset: clean slate

Every entry recorded before this date was discarded, together with the environments and runs it described (CartPole and puck-on-tilted-plane training runs, Qwen2.5-Coder-3B frozen baselines, the slosh gain sweep, the slosh frozen one- and two-turn baselines, and the first branched slosh training run). None of those results is to be cited or built on. The slosh tank is the only environment going forward; the experiment ladder restarts at the environment-validity gate described in `docs/paper.tex`.

2026-10-24 Rung 0 gate: hiding the slosh sensor and the slosh length does not make the lab necessary

`b6485c0` · *hand-written controllers only, CPU; no model was run*

Setup / motivation

The ladder restarts with a validity gate: before any model is sampled, show with hand-written controllers that the scored `policy(state)` cannot succeed without information that only the turn-1 lab can provide. Two facts about the env as shipped were established first (200 seeds per value). With the actuator gain $\kappa \in \{0.65, 0.80, 1.00\}$ hidden, a fixed PD controller with the gain constant hard-coded to 0.65 succeeds in 100% of episodes at every κ; the shipped "blind" reference is the same formula with the constant at 1.0 (0 / 50.5 / 100%), so the gap it exhibited was a tuning artifact. With the slosh length L varied from 0.3 to 1.5 m and the full state observed, a PD controller that knows nothing about L also succeeds in 100% of episodes at every L: feedback on θ does the probe's job for free.

The redesign under test therefore hid the slosh state. The scored controller observed only $(t, x, v, \text{goal}, \text{steps_left})$; θ and ω were visible only to the turn-1 lab env. The hidden parameter was L (slosh period $T_n = 2\pi\sqrt{L/g}$), with κ fixed at 1. Because θ never enters the cart update in `SloshTankEnv.step`, the scored controller's inputs are independent of L, so no controller, stateful or not, can identify L in-episode; the only blind strategy is robustness. The gate measured what robustness costs. A *blind* side (one controller for every L in the set) and an *informed* side (one controller per L, the upper bound on what a perfect turn-1 measurement could buy) searched the same five controller classes with the same budget on the same 48 search seeds: shaped bang-bang moves (none/ZV/ZVD/ZVDD/EI/two-mode shapers plus a terminal PD), free-form antisymmetric open-loop profiles with 8 segments per half, pure PD on (x, v), shaped PD with memory, and feedforward plus tracking feedback. Each job ran 800 random samples and 8×150 hill-climbing steps; winners were chosen on the search seeds and reported on 200 fresh seeds per L. The gates were fixed beforehand: G1, informed $\geq 90\%$ at every L; G2, informed mean minus blind mean ≥ 40 percentage points; G3, a hand-written two-turn solution (lab measurement, then controller) $\geq 95\%$ on fresh seeds. Twelve configurations were run (1.3 CPU-hours, 7 minutes on 11 cores).

Results

L set (m)	steps	blind per L (%)	blind mean	informed per L (%)	informed mean	gap (pp)
$\{2, 3, 4\}$	80	100 / 100 / 0	67	100 / 100 / 99	100	+33
$\{2, 3, 4\}$	90	96 / 100 / 86	94	99 / 100 / 100	100	+5
$\{2, 3, 4\}$	105	100 / 100 / 100	100	99 / 100 / 100	100	0
$\{3, 4, 5\}$	90	98 / 98 / 98	98	100 / 100 / 100	100	+2
$\{3, 4, 5\}$	105, 120, 135	100 / 100 / 100	100	100 / 100 / 100	100	0
$\{2.5, 3.5, 4.5\}$	105	99 / 100 / 100	100	100 / 100 / 100	100	0
$\{2.5, 3.5, 4.5\}$	120	100 / 100 / 100	100	100 / 100 / 100	100	0
$U[2, 4]$	90	100 / 100 / 100	100	100 / 100 / 100	100	0
$U[3, 5]$	105	100 / 100 / 82	94	100 / 100 / 100	100	+6
$U[3, 5]$	120	96 / 100 / 100	99	100 / 100 / 100	100	+1

G1 passed everywhere. G2 failed in every configuration. The blind winner was always a free-form open-loop profile, typically a short bang-bang convolved with a two-frequency shaper, whose success on the continuous ranges was 100% over uniformly drawn L. G3 passed at 100% for every L in every configuration with at least 90 steps: the lab code applied a four-step pulse and read the period off the zero crossings of θ to within 0.2% (e.g. 4.008 s measured against 4.012 s true), chose the move acceleration on lab goals disjoint from the scoring goals, and the turn-2 controller was a ZV-shaped bang-bang with the two constants filled in. At 80 steps G3 fell to 90.5% at $L = 4$.

Interpretation

The premise of the redesign holds, but an unobservable parameter is not the same as a necessary measurement. Robust input shaping over ±20% of frequency uncertainty costs little time, and the optimizer found profiles that keep the residual slosh inside the tolerance for all three periods at once. The +58 pp gap seen in the pilot came from a controller family that contained only classical single-frequency shapers; once the blind side was allowed free-form profiles it vanished. A gap appears only when the budget is squeezed below what the longest period needs, and there it also hurts the informed side (G3 at 80 steps), so it is a budget artifact rather than an information gap.

Three constraints on any future design follow. First, anything the scored controller observes that depends on the hidden quantity reveals it within a step or two, because the dynamics are deterministic; the hidden quantity must affect only variables the scored controller cannot see. Second, if the hidden quantity admits a robust compromise, the lab buys nothing; frequency uncertainty of this width admits one. Third, a scale-like parameter such as the actuator gain admits no compromise under a fixed horizon, but the env's termination on first success lets a blind open-loop plan test hypotheses in sequence: with $\kappa \in \{0.65, 0.8, 1.0\}$ a plan that stops at planned positions d, $1.25d$ and $1.54d$ succeeds under each value in turn. Success would have to be judged at the final step.

Hypotheses / next steps

1. Run the same gate on a dead-reckoning variant: the scored controller observes only $(t, \text{goal}, \text{steps_left})$, the gain (and optionally the period) is hidden, and success is evaluated at the last step with no early termination. The blind side can then be right for at most one gain value, so the gap exists by construction; the gate would confirm the informed side reaches $\geq 90\%$ and the hand-written two-turn solution $\geq 95\%$.
2. If it passes, push the two-turn solution through the real runner with the observation filter, a lab proxy that hides `env.hidden`, a lab seed disjoint from the scoring seeds, and scoring on at least four goals.
3. Check at Rung 2 that a frozen model does not already solve the lab step, since measuring a gain takes one line of code.

Open questions

Whether the no-sensor story is physically acceptable to a reviewer; whether judging success at the final step sparsifies the reward enough to hurt single-turn RL at Rung 3; whether slosh is still worth keeping once the gain carries the information gap.

Reproduce

`b6485c0`, `scripts/rung0_gate.py`: `search --n-random 800 --topk 8 --n-climb 150` over the twelve configurations listed above, and `two-turn --L 2 3 4 --steps 90`. Raw output: `docs/rung0/search_2026-10-24.log`.

2026-10-24 Rung 0 gate, second pass: dead reckoning with a hidden gain passes, provided success is judged at the final step

`3dc6581` · *hand-written controllers only, CPU; no model was run*

Setup / motivation

Following item 1 of the previous entry, the scored controller was stripped of every sensor: it observed only $(t, \text{goal}, \text{steps_left})$. The actuator gain κ was hidden (discrete $\{0.65, 0.8, 1.0\}$ or continuous $U[0.6, 1.0]$), alone with the shipped slosh length $L = 0.7\,\text{m}$ or together with a hidden L. Nothing the scored controller sees depends on either value, so the lab is the only channel, and a gain admits no robust compromise: a plan that lands on the goal under one gain lands short or long under another. One loophole remained. The shipped env ends the episode at the first step that satisfies the success test, so a blind plan can stop at the goal position implied by each gain hypothesis in turn and be credited whichever is true. A thin subclass therefore judged success at the final step only and paid the bonus only there; the shipped rule was kept as a control. The protocol was the one of the previous entry (same classes and budget on both sides, 48 search seeds, 200 fresh seeds per hidden value, gates G1–G3 fixed in advance), restricted to the open-loop classes since there is no feedback to use, plus a new multi-stop class (back-to-back shaped moves through planned positions, with an idle dwell) so the blind side could exploit the loophole wherever the rules allowed it. Six configurations were run with the final-step rule and two as the control (126 and 24 jobs; 6 minutes on 11 cores).

Results

hidden	steps	success rule	blind mean (%)	informed mean (%)	gap (pp)	G1 / G2
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	105	final step	33	100	+67	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	90	final step	33	100	+66	pass / pass
$\kappa \sim U[0.6, 1]$, $L = 0.7$	105	final step	3 (range 18)	100	+97	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L \in \{2, 3, 4\}$	105	final step	33	100	+67	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L \in \{2, 3, 4\}$	120	final step	33	100	+66	pass / pass
$\kappa \sim U[0.6, 1]$, $L \sim U[2, 4]$	105	final step	33 (range 24)	100	+66	pass / pass
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	105	first success (control)	71	99	+28	pass / FAIL
$\kappa \in \{0.65, 0.8, 1\}$, $L = 0.7$	135	first success (control)	100	100	0	pass / FAIL

Under the final-step rule the blind winner succeeded for exactly one gain value (100 / 0 / 0 across the three) in every discrete configuration and for 18% of a continuous gain range; the informed side was at 99–100% for every hidden pair, including the nine (κ, L) pairs. Under the shipped rule the blind winner was always the multi-stop class with three stops at planned positions d, $1.26d$ and $1.45d$: 13 / 100 / 100 at 105 steps and 100 / 100 / 99 at 135 steps. G3 passed at 100% for every hidden pair in all four configurations tried ($L = 0.7$ and $L \in \{2, 3, 4\}$ at 90–120 steps). The lab code recovered κ exactly from the velocity after one full-throttle step and the slosh period to within 0.3% from a four-step pulse; the turn-2 controller was the ZV-shaped bang-bang of the previous entry with the gain folded into its scaling and no terminal feedback.

Interpretation

The gap now exists by construction rather than by search: with no field sensors the scored controller cannot learn the gain, and a gain cannot be hedged, so the blind side is capped at one hypothesis (a third of a three-value set, a tolerance-width sliver of a continuous range). The measurement is trivial for the lab and useless to withhold. The control shows that the final-step rule is load-bearing and not cosmetic: given 135 steps the shipped first-success rule lets a blind plan reach 100% by visiting three candidate positions in sequence. Hiding L as well adds nothing to the gap (the gain already caps the blind side) but makes turn 2 harder and turn 1 richer, two quantities to measure in one lab episode. The continuous gain range is preferable to the discrete set: it leaves nothing to memorize and drives the blind side to a few percent.

Hypotheses / next steps

1. Env and runner changes for this variant: an observation filter so the scored `policy(state)` receives only $(t, \text{goal}, \text{steps_left})$ while the turn-1 lab env keeps the full observation; the final-step success rule with the bonus paid only at the end; L added to `HiddenParams`; a lab proxy that exposes `reset`, `step` and `observe` but not `env.hidden`; a lab seed disjoint from the scoring seeds; scoring on at least four goals.
2. Rung 0b: the hand-written two-turn solution through the real `execute_turn` path at $\geq 95\%$.
3. Rungs 1–2 as planned, with the explicit risk that a frozen model solves the one-line gain measurement from the prompt alone; if it does, the ladder tests competence rather than probing, and the paper's framing follows.

Open questions

Whether the final-step rule, which withholds the bonus until the end, sparsifies the reward enough to slow single-turn RL at Rung 3 (the shaping terms remain per step). Whether to present the task as open-loop feedforward actuation calibrated on a bench, and whether a reviewer accepts that framing. Whether L should stay hidden or be fixed and known.

Reproduce

`3dc6581`, `scripts/rung0_gate.py` with the flags `--obs dead` and `--fixed-horizon`: the `search` subcommand over the six configurations in the table, written as in `"k=0.65,0.8,1.0;L=0.7@105"`, with the budget `--n-random 800`, `--topk 8`, `--n-climb 150`; the control with `--obs dead` alone; G3 with the `two-turn` subcommand and the same flags. Raw output in `docs/rung0/`: `search_dead_fixed_2026-10-24.log` and `search_dead_early_control_2026-10-24.log`.

2026-10-24 New environment: a seeded cooling crystallizer with a hidden metastable limit passes the Rung 0 gate

`038e6d7` · *hand-written controllers only, CPU; no model was run*

Setup / motivation

The slosh tank was dropped after the two previous entries. Its hidden numbers were dynamics parameters, visible in the first steps of the final run, and the only variant that passed the gate did so by removing sensors. The rule that replaced it: the probe run is the same env with the same actions and the same readings as the final run, so the hidden quantity must be one that a probe run can reveal through the failure flag, the end-of-episode outcome or free repetition, and that a single scored episode cannot. A limit that is only found by crossing it is the cleanest case, because crossing it in the final run is the failure. The seeded batch cooling crystallizer (`scripts/seed_tasks/crystallizer_env.py`) implements it. A saturated solution carrying seed crystals is cooled from about 60 to about 20 °C; the cooling rate (jacket limit 1.5 °C/min, 20 s steps, 200 min horizon) is the only control; the supersaturation $\sigma = c/c^*(T) - 1$ drives seed growth ($G = k_g \sigma^{1.5}$) and is consumed by it; above the hidden metastable limit $\sigma_{\max} \sim U[0.08, 0.25]$ a shower of new nuclei ends the batch. The observation is $(t, T, c, L, T_{\text{target}}, \text{steps_left})$; $\sigma_{\max}$ appears nowhere except in the shower test, and two batches with limits 0.10 and 0.25 gave identical observations under identical actions until one showered. Success is reaching the target temperature equilibrated ($\sigma \leq 0.02$); reward -0.1 per step, $+120$ on success, -80 on a shower. The fastest safe schedule rides just below the limit and accelerates as the crystals grow (the classical programmed-cooling result), so the final run needs both the limit and a real controller. The protocol was that of the previous entries: five controller families (one-step-lookahead

supersaturation tracking, linear cooling, power-law cooling, piecewise-linear profile, power-law profile with a supersaturation cap), 300 random samples plus 120 hill-climbing steps from the top six, 48 search seeds, 200 fresh seeds, both sides maximizing mean env reward. Scores: success; fast success, i.e. finishing within 1.25× the time of the oracle that rides exactly at $\sigma_{\max}$; and normalized time (oracle steps over own steps, 0 on failure). Gates fixed in advance: G1 informed fast success $\geq 90\%$ at every anchor value; G2 blind fast success at least 40 pp below; G3 hand-written probe run plus final run $\geq 95\%$.

Results

hidden $\sigma_{\max}$	time wt	blind fast (%)	blind norm. time	blind reward	inf. fast (%)	inf. norm. time	inf. reward	G1 / G2 (gap)
$U[0.08, 0.25]$, anchors 0.08 / 0.165 / 0.25	0.1	7.5 (100 / 0 / 0)	0.50	91	100	1.00	91 / 108 / 110	pass / pass (+92)
$\{0.08, 0.12, 0.18, 0.25\}$	0.1	25 (100 / 0 / 0 / 0)	0.58	91	100	1.00	91 / 103 / 109 / 110	pass / pass (+75)
$U[0.08, 0.25]$ (sensitivity)	0.3	11 (0 / 0 / 0)	0.52	38	100	1.00	33 / 83 / 91	pass / pass (+89)

In both configurations the blind winner was the tracking family riding at $\sigma_t \approx 0.08$, the lowest limit in the range, which never showers and needs about 100 minutes for every batch; the informed side finished in 33 to 98 minutes depending on the limit. The open-loop families could not reach fast success even when informed ($\leq 10\%$), so feedback on the concentration is needed to ride the limit. G3 passed at 100% fast success at every anchor in both configurations: the probe run cooled at the jacket limit until the shower, then in two more batches crept up in eighths of the remaining bracket, and recovered the limit to within 0.0002 (0.004 at the top of the range); the final run rode at 98% of the recovered value.

Interpretation

The gap comes from physics and not from a narrow controller family or a withheld sensor. A limit cannot be learned without crossing it, crossing it in the final run is the failure, and feedback on every reading is allowed. Under the env reward the blind side's best policy is caution, so the probe run buys time rather than success: two to three times faster on the easy batches, which is 12 to 19 reward points out of 120 at a time weight of 0.1. Tripling the time weight raises that to about 32 points over the range but the blind side then starts to gamble ($\sigma_t \approx 0.084$, showering the 4% of batches whose limit lies below it). Compared with the slosh tank, nothing was hidden from the final run that a plant would have, and the probe's information channel, the failure flag, is one both runs share.

Hypotheses / next steps

1. Runner: the scenario carries a `hidden` dict and the templates build the hidden parameters from it; the probe-run env is wrapped so that reset, step, observe, the success and safety tests and the config are reachable but the hidden parameters are not; the probe-run seed differs from the scoring seed (same limit, different batch); a task builder with the prompt text; tests.
2. Rung 0b: the hand-written probe run and final run through the real `execute_turn` path at $\geq 95\%$.
3. Fix the time weight (0.1 or 0.3) before any training run; it sets how much of the reward the probe is worth against how much the blind side is tempted to gamble.
4. Rungs 1–2 as planned. The probe is a few lines; a frozen model may already write it, and the ladder will say so.

Open questions

Whether 600 steps per episode is acceptable for the branched-rollout cost. Whether the growth constant k_g should also vary by batch; it is identifiable from the readings of the final run, which

would show the contrast between a quantity that needs the probe run and one that does not inside a single env. Whether to add a jacket lag, which would make riding the limit harder and force a margin.

Reproduce

038e6d7, `scripts/rung0_gate_crystal.py`: the `search` subcommand over the two configurations `range:0.08-0.25` and `set:0.08,0.12,0.18,0.25` with the default budget (300 random, top 6, 120 climbing steps); the sensitivity row with the time weight set to 0.3 on the command line; G3 with the `two-turn` subcommand. Raw output in `docs/rung0/`, files `crystal_search`, `crystal_two_turn` and `crystal_search_timeweight03`, all dated 2026-10-24.

2026-10-25 The crystallizer replaces the slosh tank throughout the pipeline; Stage 0b passes through the runner

d849b0f · *hand-written probe run and final run only, CPU; no model was run*

Setup / motivation

The previous entry left the crystallizer as a standalone module next to a pipeline still wired to the slosh tank: the runner built every env with a hard-coded actuator gain, and the task files, configs, eval metric and tests were the slosh ones, so the slosh env was still needed although it had been dropped. This entry records the rewiring and the removal. A task's scenario now carries the batch seed of the final run, a different batch seed for the probe run, the hidden parameters as a dict that the runner splats into the env's `HiddenParams`, and the number of steps the time-optimal controller needs on the scoring batch. The probe run sees the env through a wrapper that forwards every public method and reading but not the hidden parameters, cannot read its own script, and runs on the probe batch. The eval metric changed from the slosh settle test to the env's own success test plus fast success (success within 1.25× the oracle time) and normalised time (oracle steps over own steps, zero on failure); fast success is the headline, since a cautious blind policy succeeds on every batch by being slow. The frozen-model script for Stages 1 and 2 was rewritten for the new task files and keeps the turn-1 mode and code per sample. The slosh env, its task files, configs, gate script, gain sweep and difficulty plan were deleted; the Stage 0 logs under `docs/rung0/` remain as the record behind the earlier entries. One inconsistency surfaced on the way: the runner calls `reset()` once after constructing the env, which under the env's seeded stream is the second batch for that seed, while the in-process helper used the first. The helper now mirrors the runner, so the stored oracle times refer to the batch the final run is scored on; the Stage 0 numbers of the previous entry were computed with the earlier helper and reproduce at their cited commit.

Stage 0b asks whether the hand-written solution survives the real execution path: the three-batch probe source from the Stage 0 gate pushed through `execute_turn` (sandbox subprocess, probe-run wrapper, probe batch), its printed bracket pasted into the final-run template, and the final policy scored with `score_sample` exactly as the trainer scores it. Pre-registered bar: fast success $\geq 95\%$. Tasks: the 24 training and 12 held-out batches plus 200 fresh batches whose limits follow the task builder's rule, 236 in all.

Results

			hand-written probe run + final run						blind cautious, same batches		
$\sigma_{\max}$ third	n	range	success	fast	norm. time	min	reward	probe err.	fast	norm. time	reward
low	78	0.080–0.136	100%	100%	0.974	0.968	99.0	0.0002	23.1%	0.674	89.7
middle	78	0.138–0.192	100%	100%	0.983	0.975	107.3	0.0002	0.0%	0.412	89.7
high	80	0.193–0.249	100%	100%	0.989	0.981	109.6	0.0002	0.0%	0.336	89.3
all	236	0.080–0.249	100%	100%	0.982	0.968	105.3	0.0002	7.6%	0.473	89.6

Gate: fast success through the real path 100% with no probe failures: pass. The 236 two-turn tasks took 1.7 s on ten workers; the env is not where the cost of a rollout lies. The full test suite (137 tests) passes, including the probe-run wrapper tests (hidden parameters unreadable, probe batch different from the scoring batch, own script unreadable) and an end-to-end test of the hand-written solution on the hardest held-out batch.

Interpretation

The numbers are the ones the in-process gate gave (probe error 2×10^{-4}, normalised time 0.97–0.99 by third), so the sandbox, the wrapper and the switch to a separate probe batch cost nothing: the pipeline carries the crystallizer end to end, and the hidden limit reaches the env only through the task file. The blind reference through the same batches is unchanged as well (fast 7.6%, normalised time 0.47), which fixes the two ends of the scale the model stages will be read against.

Hypotheses / next steps

1. Stage 1: the frozen model, one turn, no probe run; expected near the blind reference if it writes a working controller at all.
2. Stage 2: the frozen model with a probe run; read the per-sample turn-1 mode and code, not only the score, to see which probe it writes. If it already lands near the informed ceiling, the env has to get harder before any training run.
3. Fix the time weight (0.1 as shipped, or 0.3) before Stage 3; it sets how much reward the probe is worth against how much the blind side is tempted to gamble.
4. Build the stress set (limits outside the training range) that the paper's metric paragraph promises.

Open questions

Whether the probe-run wrapper is tight enough: the inner env is reachable through function introspection by a determined reader, though nothing in the reward pushes a model that way, since the honest one-batch probe already yields the limit. Whether the stress set should extend the limits downward as well, where the slowest safe batch no longer fits the 600-step horizon.

Reproduce

`d849b0f`: the task builder in `scripts/seed_tasks`, then `scripts/stage0b_two_turn.py` with 200 fresh batches and ten workers. Raw output in `docs/stage0b/`, file `two_turn_runner_2026-10-25.log`.

2026-10-25 Stages 1 and 2: the frozen model gambles at the top of the range and uses the probe run as a ritual

ebba137 · *frozen `Qwen3-4B-Instruct-2507`, no training; one A30 per stage on the HPI cluster, 18 and 21 minutes*

Setup / motivation

Stage 1 asks whether the frozen base model can write a working cooling policy from the prompt alone and how fast it is without a probe run; Stage 2 asks whether it already writes a useful probe run and uses its output. Both ran with `scripts/eval_frozen_baseline.py` on the 12 held-out batches (hidden limits 0.095 to 0.247), 8 samples per batch at temperature 0.8, 600 tokens per turn, scored exactly as the trainer scores: the final run on the scoring batch, success being the env's own test (at the target temperature, equilibrated, never showered), fast success within 1.25× the oracle time. In Stage 2 the probe run saw the probe-run env (same limit, a different batch, the limit masked) and the final turn had to be a policy; the turn-1 mode, code and tool output were kept per sample. The references on the same 12 batches: the blind cautious controller (rides at supersaturation 0.077, safe for every limit in the range) and the oracle (tracks the true limit).

Results

	ran	success	fast	norm. time	reward
Stage 1: frozen model, one turn	99.0%	12.5%	1.0%	0.075	−61.2
Stage 2: frozen model, probe run + final run	100.0%	3.1%	0.0%	0.013	−75.7
blind cautious controller	–	100%	0.0%	0.430	89.0
oracle (knows the limit)	–	100%	100%	1.000	106.8

Outcomes of the 96 Stage 1 final runs: 79 ruined (the batch showered or the seeds dissolved), 11 successes that were slow, 1 fast success, 4 timeouts, 1 sample without a valid policy. The ruined runs end after a median of 22 steps: the policy cools far too hard from the start. Of the 96 policies, 92 contain the constant 0.25, the top of the range the prompt states for the hidden limit, and 68 command full cooling somewhere. Success appears only at limits above 0.14 and reaches 50% on the batch with the highest limit (0.247); the 12 successful runs average a normalised time of 0.60.

In Stage 2 the model chose `# MODE: main` in all 96 first turns, but 83 of the 96 probe runs only reset the environment and print its initial state, the shape of the format example in the prompt; 9 step the environment a few times at full cooling, on an environment they construct themselves with a random limit rather than the one provided; 4 step gently. Only 12 tool outputs mention supersaturation, a limit or a shower at all, and none brackets the limit. The final policies then hard-code numbers read from the probe output (the probe batch's target temperature, for instance) in 93 of 96 cases, and all 96 contain the constant 0.25. Outcomes: 81 ruined, 12 timeouts, 3 slow successes, no fast success. Blind fast success is 0% on this set because its lowest limit is 0.095, above the level the cautious controller rides at.

Interpretation

The frozen model is far below the blind reference, not between blind and oracle: it reads the stated range and gambles at its top, so the batch is ruined on almost every limit. The probe run does not help because the model treats it as a format to follow, not as an experiment to run: it copies the prompt's example (reset, print the state), and the few samples that do step the environment probe a copy with a limit of their own choosing. Reading the probe output then

makes things worse, since numbers from the probe batch are pasted into a policy that runs on a different batch. The risk named in the 2026-10-24 entry, that the frozen model already writes the one-batch probe, did not materialise: none of 96 found the limit. Two things are left to learn, in order, and the scale is large: caution first (the blind optimum is worth +89 against −61 now), then the probe (+107). The prompt's example is an attractor for the probe run as it was for the policy in the discarded runs; whether to change it is a design decision, not a result.

Hypotheses / next steps

1. Decide the time weight (0.1 as shipped, or 0.3) before Stage 3. At 0.1 the gap between caution and the probe is 18 reward points; the gap between the frozen model and caution is 150.
2. Decide whether the prompt's format example should step the environment rather than only print the initial state, knowing that an opinionated example becomes an attractor under training.
3. Stage 3: single-turn GRPO; the first thing it must learn is to stop gambling at 0.25. Expected ceiling: the blind optimum.
4. Stage 4 to 6 as planned.

Open questions

Whether stating the range of the hidden limit in the prompt is right: it is what makes the cautious strategy derivable, and also what the frozen model anchors on. Whether the probe-batch numbers pasted into final policies will persist under training, where the mismatch is punished every time.

Reproduce

`ebba137`: `jobs/hpi/stage1.sbatch` and `stage2.sbatch` on the HPI cluster (A30, jobs 2782187 and 2782188), which run `scripts/eval_frozen_baseline.py` with 8 samples per task. Raw output in `docs/stage1/` and `docs/stage2/`: the job log, the summary, and the samples with code, probe output and outcome per sample.

2026-10-26 Stage 2 example ablation: the example planted the ritual, and without one the model never probes

`65bfcab` · *frozen `Qwen3-4B-Instruct-2507`, no training; three A30 jobs on the HPI cluster, 20 to 30 minutes each*

Setup / motivation

The Stage 2 entry of 2026-10-25 found that 83 of 96 probe runs only reset the environment and print its state, the shape of the format example in the prompt. This entry asks whether the example caused that, and what replaces it. The example became a config option (`prompt.example`) with three styles: *full*, the original (reset the env, print the state, then a no-op policy); *format*, the same two blocks and mode directives with a comment in place of the probe code, so there is nothing to copy; *none*, no example section, the protocol text alone. Everything else is Stage 2 as run before: 12 held-out batches, 8 samples each, temperature 0.8, 600 tokens per turn, the probe run on the probe-run env, the final turn a policy. The *full* run is a replication of Stage 2 with fresh sampling.

Results

out of 96 samples	full	format	none
took the probe run	96	79	0
turn 1 rejected (no recognised block)	0	17	59
valid final policy	95	96	37
probe copies the example (reset and print, no step)	80	31	0
probe announces it will find the limit	19	61	0
probe steps an env in a loop and checks the end flag	8	44	0
on the given env	5	0	0
on a fresh env of its own	3	44	0
probe constructs a fresh env	5	55	0
with a limit the model chose	5	39	0
and prints the limit it set	1	20	0
probe heats while talking about cooling	1	5	0
probe failed at execution	0	10	0
tool output shows a shower	0	18	0
final policy keeps the constant 0.25	94	92	37
final policy with another setpoint	1	0	0
final policy pastes probe-batch numbers	91	22	0
success / fast success	5 / 0	7 / 1	2 / 0
normalised time	0.025	0.040	0.013
mean reward	−73.8	−69.7	−150.2
probe length, median lines	9	24.5	0

The 17 rejected first turns of *format* are probe runs cut off by the 600-token budget before the closing fence: the model writes long comment preambles about what it intends to do. The 10 probe runs that failed at execution all tried `import env`. The 59 rejected first turns of *none* are correct policies written without the fence, the habit the single-turn prompt teaches; they were scored at the error floor. In *format*, 39 of the 55 fresh environments were constructed with a limit the model picked itself (0.15 is typical), 20 print the limit they set, and the fresh env is bound to the name `env`, shadowing the one provided. The 18 showers in the tool output all happened on fresh environments.

Interpretation

The example did plant the ritual: without probe code to copy, the share of reset-and-print probes fell from 80 to 31. It also does something else: it is the only reason the model takes the probe turn at all. With no example the model answered with a policy in all 96 first turns, although the protocol offered the probe run and the goal text says the limit is hidden. The model does not feel the need for an experiment; it runs one when shown one being run. Given room, it does try to measure the limit, and the experiments have the right shape (cool in a loop, watch for the end flag), but they run on the wrong object: a fresh crystallizer whose limit the model set or drew at random, which can only confirm a guess. The model does not know that a fresh env has a different limit from the one it will be scored on. And even where its probe showed a shower, no final policy took a number from the output; the setpoint stayed at 0.25. The two missing pieces are both about the given environment: that the probe must run on it, and that the number found there becomes the setpoint. Neither needs more cleverness; both can be said in the prompt without saying how to find the limit. The *format* example is the default from this commit on: it keeps the format (96 of 96 valid final policies) and removes the copying. Two operational findings: *format* probes are three times longer, and the branched trainer's 400-token turn budget would cut 45 of these 96; and the fence rule rejected 59 correct policies.

Hypotheses / next steps

1. A wording arm, same three-run protocol: say what the probe run is for and what not to do ("`sigma_max` is unknown to you; the probe run is your only chance to learn it. Use the given `env`; a new `CrystallizerEnv` draws a different random limit and tells you nothing about this task"), and replace the requirements line "run env instances, compute gains", which invites exactly the wrong experiment. Measure probes on the given env, showers on the given env, and setpoints taken from the output.
2. Accept an unfenced answer whose first non-blank line is the mode directive, in the parser `extract_code_with_mode` of `src/runner.py`, so training does not lose samples to a formality.
3. Decide the per-turn token budget for training: raise `max_turn_tokens` from 400 to the 600 used here, or let truncation pressure shorten the probes.
4. Then the time-weight decision and Stage 3.

Open questions

Whether telling the model what the probe run is for changes the paper's claim: RL would then learn how to probe, not that it should. Whether the sign error (heating while meaning to cool) and the `import env` attempts persist under training. Whether the long comment preambles survive a tighter budget or the model learns to drop them.

Reproduce

`65bfcab`: `jobs/hpi/stage2_example.sbatch` with `--export=ALL,EXAMPLE=<style>` on the HPI cluster (A30, jobs 2782452 full, 2782453 format, 2782454 none), which runs `scripts/eval_frozen_baseline.py --turns 2 --example <style>`. Compact results in `docs/stage2/` with the suffix `example-<style>`: job log, summary, samples with probe code, tool output and outcome. `docs/reproducibility/65bfcab.py` computes every number in the table from those sample files.

2026-10-26 The brief: physics in words, sigma as a reading, no source, no directives; the probe run becomes an experiment that mostly dies before it runs

`f4d95ac` · *frozen `Qwen3-4B-Instruct-2507`, no training; Stage 0b in-process, Stages 1 and 2 on one A30 each, 24 and 42 minutes*

Setup / motivation

The example ablation (entry above) showed that the frozen model takes the probe turn only because the example shows one being taken, and that given room it experiments on a crystallizer of its own making. The user's diagnosis was that the prompt had become protocol engineering, and the decision was to brief the model as an engineer would be briefed: the goal and the physics in words, no equation; what it observes and what it does not; what the probe run is for and what it offers; how the final run is scored. Three consequences followed. Without the solubility equation the model cannot compute the supersaturation, so σ became a reading in the state. With σ a reading, the shower test had to move to the reported end-of-step value: the mid-step value it used before is up to 0.03 above the reading late in a batch, so a controller that sees only readings could never have ridden the limit. And with the source out of the prompt, the probe-run object became a whitelist (reset, step, the end flag, the failure reason, the two outcome tests), the environment's source runs in a private namespace, and the static checks reject introspection, so the only crystallizer the probe code can study is the one it is given. Mode directives and

the format example are gone; turn kinds are positional and the parser accepts a fenced, bare or cut-off block. The time weight is 0.3 (a good probe is worth +53 over the blind controller, against +18 at 0.1); the error floor is −300. The oracle is now the greedy controller that holds the reading at the limit (130 steps on the 36 task batches; the blind controller takes 306). Task files were rebuilt, the hand-written solution was rewritten from the brief alone, and Stages 0b, 1 and 2 were run again. The brief is reproduced in the paper's Appendix *The brief*.

Results

Stage 0b from the brief. A three-batch probe that only resets, steps and reads σ (cool at the jacket limit until the shower, then creep up from the last safe level on two more batches) and a feedback controller with memory that holds the reading 0.003 under the level found: through the real runner, 100% fast success on 136 batches (36 from the task files, 100 fresh), normalised time 0.95 (min 0.80), median probe error 0.004, at most 0.011, mean reward +76 against +28 for the blind controller, which is fast on 4%. Simpler controllers written from the same number: full cooling until the reading hits it, then hold, showers on 21 of 36 batches with a thin margin and is fast on 33 with one step of margin; proportional control without memory is fast on 34.

Stages 1 and 2 under the brief (12 held-out batches, 8 samples each, temperature 0.8, 600 tokens per turn):

out of 96 samples	Stage 1, one turn	Stage 2, probe + final	blind	oracle
valid controller	68	84	–	–
cut off by the 600-token budget	28	12	–	–
success	14	24	100%	100%
fast success	0	2	0%	100%
ruined (median steps)	27 (23)	24 (20)	–	–
timeout	24	33	–	–
normalised time	0.068	0.144	0.414	1.000
mean reward	−160	−116	+27	+82

The probe runs of Stage 2, out of 96, against the protocol prompt's Stage 2 of 2026-10-25: step the environment 87 (16 before); cool at the jacket limit 61 (13); check the end flag or the failure reason 92 (16); run several batches 30; build a crystallizer of their own 0 (5); copy an example 0 (80). But 44 are cut off by the token budget before they can run (all end mid-code without a closing fence, at about 2400 characters, after a long commented plan), 22 crash on `state['done']` or `state['showered']`, reading the flags listed next to the state keys as state keys, and 15 print nothing. Thirteen outputs mention a shower; 5 hold the limit to within 0.01. Of the controllers, 7 reuse a number the probe printed (5 of them near the limit); when the probe had failed, several state invented results ("from probe results, shower occurred at sigma 0.22" after a probe that never ran); the one fast success is of that kind, on the batch with the highest limit. The ruined controllers cool at full rate towards a setpoint of 0.18–0.20 with no margin; most timed-out controllers never cool, or stop far above the target. Controllers read the σ reading in 68 and 83 of 96 and keep state in 68 and 49; none computes the solubility curve.

Interpretation

The brief did what the protocol prompt could not: nearly every probe is now an experiment of the right shape on the right object, and the two behaviours that defeated the earlier prompt, copying the example and building a private crystallizer, are gone. What remains is mechanical and local. Two of three probes die before producing output, from the token budget and from one misreading of the interface; the number that does get found rarely travels into the controller; and the controllers that are written either ride the setpoint with no margin or never cool. These are the things reinforcement learning is for, but half the samples of every iteration would be

spent on truncation and a wording accident, so both are worth fixing in the brief first. Rewards are not comparable with the entries of 2026-10-25 (time weight, floor and physics all changed); the counts are.

Hypotheses / next steps

1. Raise the per-turn token budget from 600 to about 1000 for evaluation and training (check the smoke config on the A30), and add one sentence to the brief: `done` is returned by `env.step`; `env.done`, `env.showered` and `env.failure_reason` are attributes of `env`, not keys of `state`. Then Stages 1 and 2 again.
2. Then Stage 3 (single-turn GRPO): the first things to learn are to stop cooling at full rate into the setpoint and to cool at all.
3. Watch, under training, whether the model keeps inventing probe results when the probe fails; the branched baseline should punish it, since the continuations of a failed probe are no better than blind.

Open questions

Whether the prior range in the brief should stay (it anchors the setpoints at 0.18–0.25 but also makes the cautious strategy derivable). Whether the hand-written controller's growth correction is something a 4B model can learn or whether proportional control with a margin (fast on 34 of 36) is the realistic ceiling.

Reproduce

`f4d95ac`: `scripts/stage0b_two_turn.py --n-random 100` (result in `docs/stage0b/two_turn_brief_2026-10-26.json`); `jobs/hpi/stage1.sbatch` and `stage2.sbatch` on the HPI cluster (A30, jobs 2782627 and 2782628), which run `scripts/eval_frozen_baseline.py --turns 1|2 --n-samples 8`. Compact results in `docs/stage1/` and `docs/stage2/` with the suffix `brief_2026-10-26`: job log, summary, samples with probe code, tool output, controller and outcome.

2026-11-01 Stage 0 rebuilt on the current environment, with the provenance of every number; the gate passes on the second run

`2edeba7` · *hand-written controllers only, CPU; two cluster jobs of 32 CPUs, 282 s and 279 s; no model was run*

Setup / motivation

The user's decision: a clean slate for Stage 0. The earlier gate had been run on the previous environment and carried over by argument; its results are discarded, nothing is kept for compatibility, and reproducibility is the first requirement: every number names the commit, the job and the log that produced it. The full account is `docs/stage0_gate.tex`, whose tables are rendered from the result JSON by `scripts/stage0_render.py`; this entry is the dated summary.

What was built. The environment gains `peek(action)`, the exact one-step preview of the reported σ, by splitting the step's arithmetic into a pure function both use; the oracle and the tracking family ride on it by bisection, and the new oracle reproduces the previous one step for step on 65 batches. The references are the rider at the limit (oracle) and the rider at the bottom of the range (cautious blind); the closed-form pre-growth tracker is gone. Task limits are drawn uniformly within thirds of the range so every set covers it evenly, and the held-out

set grows from 12 to 48 tasks (16 per third); Stages 1 and 2 must be re-run on it. The gate is one script, `scripts/stage0_gate.py`: five families (track, linear, power, hybrid, pwl) with knobs in the unit cube; a blind side tuned across limits and an informed side per anchor (0.08, 0.165, 0.25); 300 random vectors and six survivors climbing for 120 rounds on 48 search batches; scoring on 200 fresh batches by fast success against the task files' oracle, batches drawn as the runner draws them. Stage 0b is folded in as G3: the hand-written readings-only probe and controller (`scripts/stage0_solution.py`, four probe variants) run through `execute_turn` and `score_sample`, the sandbox path training uses, at every anchor and over the range, and the same solution, the oracle and both blind controllers are scored on the 48 held-out tasks as the references the model stages are read against. `src/provenance.py` embeds commit, tree state, command, host and Slurm job in every result and refuses to run without a commit; the cluster copy has no `.git`, so the job receives it via `sbatch --export`. Decisions taken from the todo list: the 48-task stratified evaluation set (item 5); no stress set, the range stays what the brief states (item 9); both blind references reported, cautious and reward-optimal (item 8); per-task oracle and informed rows for the informed frozen arm (item 6).

Results

Run 1 (commit `fc23a2b`, job 2978483). G1 pass: the track family tuned at each anchor is fast on every batch. G2 pass: the best blind controller, track at level 0.1016, is fast on 10.5% of range batches, 90 points below the informed side; it showers 23 of 200 batches, all in the low third. G3 fail at the 0.08 anchor: 84.0% fast there, 100% at 0.165 and 0.25, 99.5% over the range. The probe brackets the limit to about one step of full cooling (median error 0.005, maximum 0.011) and the controller rode a further 0.003 under that bound; at the bottom of the range that shortfall cost more than 1.25× the oracle's time on a sixth of the batches.

Run 2 (commit `2edeba7`, job 2978644). One change: the controller's setpoint is the larger of the probe's lower bound and 0.08, minus the margin, since the brief states the limit is at least 0.08. The search side is identical to run 1 (same code, seed and batches). G3 pass: 100% fast at every anchor and over the range, no probe failure, normalised time 0.950 over the range (minimum 0.829, low third). All three gates pass.

Reference, 48 held-out tasks	Steps	Reward	Success	Fast	Norm. time
oracle (rides at the limit)	134.3	79.7	100%	100%	1.000
cautious blind (rides at 0.08)	272.0	38.4	100%	4.2%	0.493
searched blind (track at 0.1016)	176.5	46.2	89.6%	10.4%	0.563
probe run + controller	143.1	77.1	100%	100%	0.951

Probe variants over the range (each followed by the same controller): three passes and one pass at the jacket limit are indistinguishable (median error 0.0049 against 0.0050, both 100% fast, normalised time 0.950), so the creep passes as written tighten nothing; a creep of 0.005 per step halves the median error but is loose by up to 0.03 at high limits; a creep of 0.001 per step fails there outright (77.5% fast, maximum error 0.11).

Interpretation

The gap comes from physics, with the strongest blind side the families can offer: feedback on the reading and the exact lookahead were allowed on both sides, and only the two feedback families ride the limit even when informed. Under the current reward the best blind policy gambles: it rides above the bottom of the range, loses the batches below its level and is still fast on one in ten. So the model stages have two blind references, the cautious one that never fails and the searched one that a reward-trained controller without a probe (arm C) can reach at most, and only by gambling, which the success column exposes. A probe that crosses the limit once, read

correctly, is the whole value of the probe run; refinement buys little, and a probe that creeps too slowly learns nothing because the crystals consume the supersaturation faster than it adds it. The hand-written solution from the brief alone lands within three reward points of the oracle.

Hypotheses / next steps

1. Re-run Stages 1 and 2 on the 48-task evaluation set; the 12-task results are stale.
2. Rewrite the paper's Stage 0 subsection and appendix from `docs/stage0_gate.tex` (its last section says what goes where).
3. Write the Stage 3 gate from the references: arm C against the searched blind controller's +46.2, with its success rate as the gambling check, and the cautious +38.4 as the no-failure line.
4. Close todo items 5, 8 and 9 and part of 6; add the Stage 1 and 2 re-run as an item.

Open questions

Whether a second probe pass that holds the reading at the lower bound by feedback, rather than creeping, would tighten the bracket; the gate does not need it. Whether a readings-only family belongs in the search, to measure the informed ceiling without the lookahead. Whether gambling by a trained controller should be penalised in the reward or only reported.

Reproduce

Run 2: `2edeba7`, `hpc push . lora4vr`, then `hpc submit lora4vr jobs/hpi/stage0.sbatch --export=ALL,LORA4VR_GIT_COMMIT=2edeba7c173ce738e608d56eb7a575122e4a02bd` (job 2978644, gx07), `hpc pull lora4vr/results/stage0_2978644 docs/stage0/`, `scripts/stage0_render.py docs/stage0/stage0_2978644 --out docs/stage0/tables`. Run 1: `fc23a2b`, the same with job 2978483, rendered to `docs/stage0/tables_run1`. Each result directory holds the job log, `stage0_summary.txt`, `stage0_results.json` with its provenance block and `stage0_rows.json` with every per-batch row. Locally, `scripts/stage0_gate.py --out <dir> --workers <n>` runs the same thing, `--quick` as a smoke test; `tests/test_stage0_gate.py` covers the pieces.

2026-11-01 Stages 1 and 2 re-run on the 48-task set: the 4B base against Qwen3-8B; a sandbox bug fixed on the way

`bea2bf8` · *`Qwen3-4B-Instruct-2507` and `Qwen3-8B` in non-thinking mode, frozen; four A30 jobs of 47 to 70 minutes; no training*

Setup / motivation

Stage 0's rebuild moved the held-out set to 48 tasks (16 per third of the limit's range), so the frozen-model stages had to be re-run; the user asked for a larger model beside the 4B base and chose `Qwen3-8B` with thinking switched off (`enable_thinking=False` in the chat template, as its model card instructs). Four runs: Stage 1 (one turn, no probe section) and Stage 2 (the two-turn protocol with the optional probe run) for each model, 4 samples per task at temperature 0.8, top-p 0.95, 1000 new tokens per turn, the brief with the two quench logs and the stated range. `scripts/eval_frozen_baseline.py` now embeds the provenance block in its summary and the job scripts take the model, the sample count and the commit; `scripts/frozen_render.py` renders the result JSON into the tables of `docs/stage12_frozen.tex`, the full account.

A first round (commit `9a83f4e`, jobs 2979823, 2979824, 2979828, 2979829) exposed a bug in the scoring sandbox: the final run returned its result as the whole stdout of the subprocess parsed as JSON, so a controller that printed anything was floored at −300 ("Output parse error"). The brief does not forbid printing in the controller, and the 8B prints far more often than the 4B (38 of 192 Stage 2 controllers against 5), so the bug cost it 11 of 192 samples in Stage 2 and 1 in Stage 1, and would have taught a trained policy never to print. Commit `bea2bf8` keeps the real stdout for the one result line, sends the controller's prints to a sink, and reads the last line; the new test fails on the old runner. All four runs were redrawn on the fixed commit (round 2, jobs 2980889, 2980890, 2980891, 2980892; the numbers below). Round 1's summaries and logs are kept under `docs/stage12/superseded/`; its samples remain on the cluster.

Results

Run	Model	Valid	Success	Fast	Norm. time	Reward	Ride ratio	Probe rate	Est. error
stage1-4b	iter_000	91.1%	27.1%	1.0%	0.156	−84.3	0.75	–	–
stage2-4b	iter_000	98.4%	23.4%	0.0%	0.119	−65.7	0.79	6.2%	–
stage1-8b	qwen3-8b	94.8%	5.7%	0.5%	0.032	−207.4	0.30	–	–
stage2-8b	qwen3-8b	98.4%	8.9%	1.0%	0.047	−191.8	0.50	5.7%	–
cautious blind reference	–	–	–	4.2%	0.493	38.4	–	–	–
oracle	–	–	–	100.0%	1.000	79.7	–	–	–

	stage1-4b	stage2-4b	stage1-8b	stage2-8b
samples that did not run	22	4	91	90
syntax error (reply cut off or malformed)	16	3	9	1
forbidden name at validation	1	0	1	2
reads state['done'], ['showered'] or ['failure_reason']	0	0	7	33
calls env in the controller	0	1	0	21
undefined name (NameError)	3	0	49	16
missing state key (KeyError)	0	0	17	5
local used before assignment	2	0	6	4
sandbox output parse error (fixed in bea2bf8)	0	0	0	0
other runtime error	0	0	2	8
controllers that call env.reset or env.step, of 192	0	6	0	37
controllers that read state['done'], ['showered'] or ['failure_reason'], of 192	0	0	7	35
controllers that print, of 192	0	5	11	38

Both frozen models sit far below the cautious blind controller (+38): the 4B at −84 (one turn) and −66 (two turns), the 8B at −207 and −192; fast success is at most one sample in a hundred. The 8B is worse on every column because 91 and 90 of its 192 samples never run: undefined names (`L` never read from the state, an `initial_L` never set), `state['done']` read although the brief states it is not a state key, and in Stage 2 `env` called inside the controller although the brief states the controller has none. Neither model uses the probe run: 12 and 11 first replies of 192 are probe code, every one cut off by the 1000-token budget and rejected by the sandbox, so no probe ran and no probe output holds a number near the limit. The 4B's Stage 2 gain over Stage 1 comes from shorter replies (cut-offs 17.7% to 3.6%), not from information: its success rate falls from 27% to 23%. By third, the 4B succeeds on about half the high-limit batches and almost none of the low-limit ones (a fixed moderate cooling rate); the 8B mostly times out. Round 1's numbers agree within sampling noise (4B −85 and −53, 8B −209 and −195).

Interpretation

The frozen base is the floor the training stages start from, and it is the whole scale below the blind reference. The larger model does not help: in non-thinking mode it writes longer code with more invented names and more breaches of the brief's stated rules, and it gives no better start at twice the memory. The probe rate of a frozen model is about six percent, and it is measured

with the token budget as a confound: no probe reply the frozen models write fits in 1000 tokens, so "does the frozen model probe" cannot be separated from "can its probe run".

Hypotheses / next steps

1. Decide the per-turn token budget before Stage 3 (todo item 2): raise it so a probe reply can run, or keep 1000 and state the confound. The user's call; it changes the protocol.
2. Train the 4B base (Stage 3 onwards); drop the 8B.
3. Rewrite the paper's Stage 1 and 2 status paragraphs from `docs/stage12_frozen.tex` together with the Stage 0 rewrite; the paper still carries the 12-task numbers of 2026-10-26.

Open questions

Why the Stage 2 brief draws shorter controller replies from the 4B than the Stage 1 brief (median 2 086 against 2 999 characters); whether the 8B in thinking mode would respect the brief's rules, at a cost in tokens the protocol does not budget for.

Reproduce

bea2bf8, `hpc push . lora4vr`, then one job per run, with `MODEL_PATH=models/hf_base/iter_000` for the 4B base and `models/hf_base/qwen3-8b` for the 8B (`hf download Qwen/Qwen3-8B` on the run node):

```
hpc submit lora4vr jobs/hpi/stage1.sbatch --job-name=stage1-<tag> \
    --export=ALL,LORA4VR_GIT_COMMIT=bea2bf8b366b8a5551b5b3a9a86baaacf3b9d242,MODEL_PATH=<path>,N_SAMPLES=4
hpc submit lora4vr jobs/hpi/stage2.sbatch --job-name=stage2-<tag> \
    --export=ALL,LORA4VR_GIT_COMMIT=bea2bf8b366b8a5551b5b3a9a86baaacf3b9d242,MODEL_PATH=<path>,N_SAMPLES=4
hpc pull lora4vr/results/<jobname>_<jobid> docs/stage12/
uv run python scripts/frozen_render.py --out docs/stage12/tables \
    stage1-4b=docs/stage12/stage1-4b_2980889 stage2-4b=docs/stage12/stage2-4b_2980890 \
    stage1-8b=docs/stage12/stage1-8b_2980891 stage2-8b=docs/stage12/stage2-8b_2980892
```

Jobs 2980889 (stage1-4b), 2980890 (stage2-4b), 2980891 (stage1-8b), 2980892 (stage2-8b), all on gx17v1. Each result directory holds the job log, `stage<N>_summary.json` with its provenance block and per-task rows, and `stage<N>_samples.jsonl` with every sample's transcript, probe code, tool output, controller and outcome. The sandbox bug is pinned by `tests/test_runner_modes.py`, test `test_policy_that_prints_is_scored_normally`.